JavaFX

One of the advanced topics we decided to work with is JavaFX which we used to create the graphical user interface of our application. We used it instead of Java Swing because it offered us more customisation options, especially through CSS styling and allowed us to design our interfaces using Scene Builder. These tools helped us create more flexibility in the appearance of the game. Working with JavaFX gave us the opportunity to learn about GUI development and how different components of an application can interact with each other.

During the implementation, we learned how JavaFX uses components such as Stage, Scene, and controls to display and organise an application window. We used FXML to define the layouts and separate it from the main logic, while the Java controllers handled user interaction and connected the interface to the game logic. This helped us understand how separating the presentation from the functionality can make the application easier to organise, maintain, and review.

One of the main challenges we encountered was connecting the interface to the game logic while ensuring that the displayed maze remained consistent with the robot's actual state. For example, executing a command required updating the robot's position or direction and then rendering the maze again so that the user could see the result. We also used JavaFX animation through Timeline and KeyFrame to execute commands over time rather than all at once. This taught me how GUI applications can use event-driven programming to respond to user actions and perform timed updates and how to properly use event handlers.

Another difficulty that we encountered was understanding how JavaFX handles its user interface thread and event processing. Operations such as displaying dialogs must be handled carefully when animations or layout updates are in progress. Troubleshooting these issues helped me better understand the execution model of a GUI application and the importance of keeping interface updates consistent with the application's state.

Path-Searching Algorithms

Another one of the main topics covered while making our game RoboMaze was the use of path-searching algorithms, specifically Breadth-First Search (BFS) for finding the

shortest path and Depth-First Search (DFS) for generating the maze. We thought this topic is relevant to our project since these algorithms fit the concept of solving a maze and we considered the use of data structures such as lists, queues, stacks to help us learn how we can adapt different algorithms to a specific problem, and prepare us for upcoming courses.

Before starting to work on the implementation, we spent time properly understanding the uses and applications of these algorithms, including the bases of graph theory, and the differences between algorithms.

We decided to use BFS for pathfinding since it explores neighboring positions level by level until it reaches the target, selecting that path and adding it to a list. This path has the shortest number of steps, without taking into account the number of turns. While implementing this I learned about how queues work and how LinkedLists can implement a queue. The First In First Out property was extremely useful, because it allowed the algorithm process positions as they are discovered. We also had to keep track of visited positions and the previous position from which each point was reached, allowing us to reconstruct the path once the target was found. The main difficulties with BFS were handling neighboring positions correctly and managing the queue to add and remove new neighbours.

A challenge that we had was calculating the number of direction changes. We did this by analyzing three consecutive points from the fastest path returned by the BFS and comparing their coordinates to determine whether there was a direction change. Although the idea was pretty clear from the start, comparing the three points in order to determine a change of direction, it was a bit more difficult to translate it into code. This made us understand how a relatively simple idea requires intensive reasoning when implementing it.

We decided to use DFS for generating the maze. We found it appropriate to use it, since it required exploring a path, and to backtrack when it reaches a dead end to an earlier position without any explored cells. While working on it, we learned about the Stack data structure, which we used to store positions on the current path and to backtrack easier. Our biggest challenge during its development was creating the corridors through which the robot can pass and to preserve the maze structure. We had to ensure that we didn't create openings that were too large or walls over one tile thick. To do this, we had to analyze how neighboring cells are selected and how the dimensions affect the generation process.